

Formattazione dell'output

Formattazione dell'output

Come presentare i dati in modo leggibile e ordinato.

Perché formattare l'output?

Stampare dati grezzi funziona, ma non è sempre leggibile. Formattare l'output significa presentare le informazioni in modo chiaro: numeri con il numero giusto di cifre decimali, testo allineato in colonne ordinate.

Le f-string: il metodo consigliato

Le **f-string** (disponibili da Python 3.6) sono il modo più comodo e leggibile per costruire stringhe con variabili. Si scrivono mettendo `f` prima delle virgolette:

```
nome = "Alice"
eta = 16
print(f"Mi chiamo {nome} e ho {eta} anni.")
# Mi chiamo Alice e ho 16 anni.
```

Dentro le parentesi graffe puoi scrivere qualsiasi espressione Python:

```
a, b = 5, 3
print(f"{a} + {b} = {a + b}")
print(f"Il quadrato di {a} è {a**2}")
```

Controllare i decimali

Per i numeri decimali puoi specificare quante cifre mostrare dopo la virgola:

```
pi = 3.14159265
print(f"{pi:.2f}")    # 3.14    – 2 cifre decimali
print(f"{pi:.4f}")    # 3.1416  – 4 cifre decimali
print(f"{pi:.0f}")    # 3       – nessuna cifra decimale
```

La sintassi è `{valore:.Nf}` dove **N** è il numero di cifre decimali.

Percentuali

```
percentuale = 0.75
print(f"{percentuale:.0%}")    # 75%
print(f"{percentuale:.2%}")    # 75.00%
```

Python moltiplica per 100 e aggiunge il simbolo `%` automaticamente.

Separatore delle migliaia

Per i numeri grandi, puoi aggiungere la virgola come separatore:

```
numero = 1234567
print(f"{numero:,}")    # 1,234,567
```

Allineare il testo in colonne

Quando vuoi creare tabelle con colonne ordinate, puoi specificare la larghezza e l'allineamento:

```
print(f"{'Sinistra':<15}|")    # allineato a sinistra – 15 caratteri
print(f"{'Centro':^15}|")      # centrato
print(f"{'Destra':>15}|")      # allineato a destra
```

Output:

```
Sinistra  |  
  Centro  |  
    Destra|
```

Opzioni di `print()`

La funzione `print()` ha due opzioni utili:

sep — il testo che viene messo tra gli argomenti (default: uno spazio):

```
print("mela", "banana", "uva")           # mela banana uva  
print("mela", "banana", "uva", sep=", ")  # mela, banana, uva  
print("mela", "banana", "uva", sep=" | ") # mela | banana | uva
```

end — il testo aggiunto alla fine (default: va a capo `\n`):

```
print("Ciao", end=" ")  
print("mondo!")          # Ciao mondo! — sulla stessa riga
```

Utile per stampare più elementi sulla stessa riga in un ciclo:

```
for i in range(5):  
    print(i, end=" ")  
print() # va a capo alla fine  
# Output: 0 1 2 3 4
```

Esempio: una tabella formattata

```
studenti = [  
    ("Alice", 16, 8.5),  
    ("Bob", 15, 7.0),  
    ("Carlo", 17, 9.2),  
]  
  
# Intestazione della tabella  
print(f"{'Nome':<10} {'Età':>5} {'Media':>8}")  
print("-" * 25)  
  
# Righe della tabella  
for nome, eta, media in studenti:  
    print(f"{'nome':<10} {'eta':>5} {'media':>8.1f}")
```

Output:

Nome	Età	Media
Alice	16	8.5
Bob	15	7.0
Carlo	17	9.2